Difussion-IBR

Varun Moparthi
https://github.com/Elonian/Diffusion-IBR

1 Introduction

Novel view synthesis aims to recover photorealistic images of a scene from unseen camera poses and has been transformed by neural rendering methods such as Neural Radiance Fields (NeRF) and 3D Gaussian Splatting (3DGS) [1, 2]. Despite their remarkable fidelity, both families remain vulnerable when supervision is weak or the query camera moves beyond the observed trajectory. In such settings, the reconstruction problem becomes severely underconstrained, and rendered views often exhibit floaters, broken geometry, texture smearing, duplicated structures, and inconsistent appearance across viewpoints. These failure modes are especially pronounced under sparse view capture, large baseline extrapolation, and real world data with imperfect calibration, lighting variation, or incomplete scene coverage.

A large body of work has attempted to reduce these artifacts by injecting stronger priors into the reconstruction pipeline. Geometry oriented methods regularize training with unseen view constraints, sparse depth, or frequency priors to reduce ambiguity and stabilize optimization [3, 4, 5, 6]. Cross scene methods such as pixelNeRF and MVSNeRF learn dataset level scene priors that enable few view generalization, while restoration based approaches such as NeRFLiX directly enhance rendered outputs using neighboring reference views [7, 8, 9]. However, deterministic priors often remain conservative in severely ambiguous regions and tend to smooth over details that must be hallucinated rather than merely interpolated.

This limitation has motivated the use of generative priors. Early work such as GANeRF introduced adversarial supervision to improve local realism while preserving 3D consistency [10]. Diffusion based methods brought even stronger image statistics to 3D tasks, ranging from diffusion guided 3D optimization and text to 3D generation to viewpoint conditioned and multi view image synthesis [11, 12, 13, 14]. More directly related to sparse view and extrapolated rendering, ReconFusion, Deceptive NeRF/3DGS, and 3DGS Enhancer show that diffusion priors can improve underconstrained reconstructions either by regularizing 3D optimization or by converting artifact prone renders into pseudo observations that are distilled back into the scene representation [15, 16, 17]. Yet these methods reveal a practical trade off: stronger task adaptation usually requires fine tuning on curated 3D data, whereas frozen or training free priors preserve broader generalization but often sacrifice fidelity or rely on heavier video diffusion pipelines.

2 Related Work

Geometry and reconstruction priors. After NeRF and 3DGS, much of the literature focused on reducing reconstruction ambiguity in sparsely observed scenes by strengthening geometric supervision during training. RegNeRF regularizes geometry and appearance at unseen viewpoints to stabilize sparse input optimization, DS NeRF exploits sparse SfM depth as inexpensive supervision, FreeNeRF shows that simple frequency and near camera density regularization already provides strong few shot gains, and SparseNeRF relaxes the need for metric depth by distilling only local depth rankings from noisy observations [3, 4, 5, 6]. These methods significantly improve consistency under weak supervision, but they mainly reduce ambiguity through regularization rather than explicit synthesis of missing content. As a result, they remain limited when artifacts arise from genuinely unseen geometry or appearance far outside the support of the input views.

Cross scene priors and restoration based enhancement. A parallel direction learns priors across many scenes instead of optimizing each scene independently from scratch. pixelNeRF conditions

radiance fields on image features and performs feed forward reconstruction from one or a few images, while MVSNeRF incorporates multi view stereo cost volumes for geometry aware generalization from sparse inputs [7, 8]. Rather than predicting a full scene representation from scratch, NeRFLiX treats neural renders as degraded observations and learns a NeRF agnostic restorer with degradation aware inter viewpoint aggregation [9]. This line of work is especially relevant because it reframes artifact removal as image restoration. However, because these methods are largely deterministic, they often average over multiple plausible solutions and therefore produce conservative or overly smooth results in severely underconstrained regions.

Generative priors for novel view synthesis. Generative models introduce substantially stronger natural image priors into novel view synthesis. GANeRF employs scene specific adversarial supervision to improve local realism while preserving 3D consistency in the underlying radiance field [10]. Diffusion based methods broaden this idea considerably. DreamFusion shows that a pretrained 2D diffusion model can supervise 3D optimization without paired 3D data, Zero 1 to 3 learns viewpoint conditioned image diffusion from a single observation, SyncDreamer models the joint distribution of multiple views to improve cross view coherence, and MVDream learns a multi view diffusion prior that is largely agnostic to the downstream 3D representation [11, 12, 13, 14]. These works demonstrate that generative priors can hallucinate unseen content more plausibly than purely geometric regularizers. Still, many were developed for object centric generation or depend on iterative denoising, which makes direct deployment inside scene scale reconstruction pipelines expensive.

Diffusion assisted reconstruction and pseudo observation refinement. More recent methods use diffusion not only to generate images, but to improve an already reconstructed 3D model. ReconFusion regularizes NeRF optimization with a diffusion prior evaluated at novel camera poses, directly coupling the generative model to the reconstruction objective [15]. Deceptive NeRF/3DGS follows a different strategy: it converts noisy renders from a coarse sparse view reconstruction into photorealistic pseudo observations and progressively densifies the training set, using uncertainty to reconcile generated views with real inputs [16]. 3DGS Enhancer extends this philosophy to unbounded Gaussian splatting by leveraging video diffusion priors, reframing view consistency as temporal consistency, and fine tuning the 3DGS model using enhanced novel views [17]. This family is the closest precursor to my project, because it treats diffusion as a repair prior for 3D reconstruction rather than as a stand alone renderer.

From heavy diffusion guidance to practical extrapolation enhancement. The latest literature exposes a central trade off between fidelity, efficiency, and generalization. ViewExtrapolator shows that video diffusion priors can be used in a training free manner for novel view extrapolation, while StreetCrafter demonstrates that controllable video diffusion with LiDAR based conditioning can enlarge the valid rendering range in street scenes and even support scene editing [18, 19]. Difix3D+ moves toward efficiency by fine tuning a single step diffusion model as a fast artifact fixer and integrating it both during progressive 3D refinement and at render time as a neural enhancer [20]. FreeFix moves in the opposite direction on the adaptation axis: it keeps the diffusion model frozen, interleaves 2D and 3DGS refinement along a trajectory of extrapolated views, and replaces opacity based guidance with a Fisher information based confidence map that better localizes unreliable regions [21]. Together, Difix3D+ and FreeFix define two complementary strategies for diffusion assisted novel view synthesis: one emphasizes task adapted, high fidelity single step repair, while the other emphasizes training free generalization and targeted guidance.

3 Method

In this project, we study three closely related pipelines on five DL3DV scenes: (i) a vanilla 3D Gaussian Splatting (3DGS) baseline, (ii) 3DGS enhanced with the released Difix model following the progressive refinement strategy of Difix3D, and (iii) FreeFix, a fine tuning free diffusion assisted refinement method. All three pipelines start from the same core idea: we first optimize a scene specific 3DGS representation from the available training views, and then improve difficult extrapolated views by injecting a 2D image prior back into the 3D representation.

3.1 Common 3DGS Formulation

We represent a scene using a set of anisotropic 3D Gaussians

$$\mathcal{G} = \{(\boldsymbol{\mu}_i, \mathbf{q}_i, \mathbf{s}_i, o_i, \mathbf{c}_i)\}_{i=1}^{N}, \tag{1}$$

where $\boldsymbol{\mu}_i \in R^3$ is the Gaussian center, $\mathbf{q}_i$ is the rotation, $\mathbf{s}_i$ is the scale, o_i is the opacity, and $\mathbf{c}_i$ denotes the color parameters. The covariance of each Gaussian is written as

$$\boldsymbol{\Sigma}_i = \mathbf{R}(\mathbf{q}_i)\,\mathrm{diag}(\mathbf{s}_i^2)\,\mathbf{R}(\mathbf{q}_i)^\top. \tag{2}$$

Given a camera view v, the renderer projects the Gaussians into the image plane and composites them in front to back order. I write the rendered color at a pixel p as

$$\hat{\mathbf{C}}(p) = \sum_{i=1}^{N} T_i(p)\,\alpha_i(p)\,\mathbf{c}_i(p), \qquad T_i(p) = \prod_{j<i} (1\alpha_j(p)), \tag{3}$$

where $\alpha_i(p)$ is the projected opacity contribution of the i th Gaussian and $T_i(p)$ is the accumulated transmittance before that Gaussian. Using this renderer, the image produced at viewpoint v is denoted by

$$\hat{I}_v = \mathcal{R}(\mathcal{G}, v). \tag{4}$$

Let the real training set be

$$\mathcal{D}_{\text{real}} = \{(I_k, v_k)\}_{k=1}^{M}. \tag{5}$$

For the vanilla 3DGS baseline, we optimize the Gaussian parameters only from the real views using a photometric loss of the form

$$\mathcal{L}_{\text{3DGS}} = (1\lambda_{\text{ssim}}) \left\| \hat{I}_{v_k} I_k \right\|_1 + \lambda_{\text{ssim}} \left(1\text{SSIM}(\hat{I}_{v_k}, I_k)\right). \tag{6}$$

In practice, this is the core loss used by the gsplat style training code, and it forms the common starting point for both Difix based refinement and FreeFix based refinement.

3.2 Vanilla 3DGS Baseline

my first system is a standard scene wise 3DGS model trained only on the original views. This baseline is important for two reasons. First, it gives us a clean reference point for how much extrapolated view quality can be achieved from geometry and photometric fitting alone. Second, both Difix3D and FreeFix are designed to *refine* an already reconstructed 3DGS model rather than replace the reconstruction stage entirely. In other words, the baseline 3DGS model is the shared initialization for all later refinement experiments.

After training, we evaluate the baseline directly on extrapolated or otherwise difficult novel views. These outputs typically remain sharp near the training cameras, but they often degrade in underconstrained regions, where we observe floaters, duplicated structures, local blur, or texture inconsistencies. The remaining two methods are designed specifically to repair these failure modes.

3.3 3DGS + Difix3D Refinement

3.3.1 Overview

my Difix based pipeline follows the refinement idea of Difix3D, but with one important practical difference: we do *not* retrain the diffusion model ourselves. Instead, we reuse the released pretrained Difix checkpoint and plug it into the progressive 3D refinement loop. This choice is appropriate for a project report because my goal is to study how the released artifact fixer improves a standard 3DGS reconstruction when used exactly in the intended way.

We denote the pretrained artifact fixer by $\mathcal{F}_\phi$. Starting from a trained baseline 3DGS model $\mathcal{G}^{(0)}$, we periodically render difficult views, refine them with Difix, and then feed the refined images back into 3DGS training as pseudo observations. The key idea is that the diffusion model corrects 2D artifacts, while the 3DGS model absorbs these corrections into a more consistent 3D representation.

3.3.2 Reference conditioned one step artifact fixing

For a target pose v, we first render the current 3DGS model:

$$I_v^{(t)} = \mathcal{R}(\mathcal{G}^{(t)}, v). \tag{7}$$

We then select a nearby reference image $I_{r(v)}$ from the original training set using nearest pose assignment. The refined view is obtained by the released Difix model:

$$\tilde{I}_v^{(t)} = \mathcal{F}_\phi\Big(I_v^{(t)}, I_{r(v)}; p_{\text{difix}}\Big), \tag{8}$$

where the text prompt is the fixed phrase

$$p_{\text{difix}} = \texttt{"remove degradation"}. \tag{9}$$

In the released implementation, this refinement is carried out with exactly one denoising step, timestep 199, and guidance scale 0.0. Since the model is a single step diffusion model, the fixer behaves like a fast image to image restoration operator rather than a long iterative generator.

3.3.3 Progressive pseudo view updates

Directly fixing only the final rendered views would improve single images, but it would not guarantee multi view consistency. To address this, we follow the Difix3D strategy of progressively adding refined pseudo views back into the 3D optimization.

Let $\mathcal{V}^{(t)}$ denote the current set of novel poses to be refined. At each scheduled refinement stage, we:

1. render the current 3DGS model at poses in $\mathcal{V}^{(t)}$,
2. refine each rendered image using Difix and its nearest reference view,
3. create a pseudo dataset
$$\mathcal{D}_{\text{pseudo}}^{(t)} = \Big\{(\tilde{I}_v^{(t)}, v) \ : \ v \in \mathcal{V}^{(t)}\Big\}, \tag{10}$$
4. continue training 3DGS on both the original and pseudo views.

Conceptually, the paper describes this as a gradual pose perturbation process in which the novel poses are moved toward the harder target poses over multiple refinement rounds. In my implementation, we follow the released gsplat version of this idea: the code maintains a current novel trajectory and shifts it toward the validation poses before running Difix again. This makes the refinement incremental instead of adding all pseudo views at once.

3.3.4 Training objective after pseudo view insertion

After adding refined views, the 3DGS optimizer sees both real and generated supervision. We write the joint objective as

$$\mathcal{L}_{\text{joint}} = \mathcal{L}_{\text{real}} + \lambda_{\text{novel}}\, \mathcal{L}_{\text{pseudo}}, \tag{11}$$

where

$$\mathcal{L}_{\text{real}} = \sum_{(I,v)\in\mathcal{D}_{\text{real}}} \mathcal{L}_{\text{3DGS}}(\mathcal{R}(\mathcal{G}, v), I), \tag{12}$$

and

$$\mathcal{L}_{\text{pseudo}} = \sum_{(\tilde{I},v)\in\mathcal{D}_{\text{pseudo}}} \mathcal{L}_{\text{3DGS}}(\mathcal{R}(\mathcal{G}, v), \tilde{I}). \tag{13}$$

Pseudo view updates are intentionally downweighted to avoid letting imperfect generated images dominate training. The code also mixes batches from real and pseudo data instead of switching completely to generated views. This makes the refinement more stable.

3.3.5 What we inherit from the released Difix model

Although we do not retrain Difix in my project, it is still useful to summarize the model we are using. Difix is built on top of SD Turbo and is designed as a reference conditioned artifact fixer. The paper fine tunes it with a frozen VAE encoder and a LoRA adapted decoder, and supervises it using a weighted combination of an L_2 reconstruction term, an LPIPS perceptual term, and a Gram matrix style term:

$$\mathcal{L}_{\text{Difix}} = \lambda_2 \, \|\hat{I} I^\star\|_2^2 + \lambda_{\text{lpips}} \, \text{LPIPS}(\hat{I}, I^\star) + \lambda_{\text{gram}} \, \mathcal{L}_{\text{gram}}(\hat{I}, I^\star). \tag{14}$$

$\mathcal{F}_\phi$ is kept fixed and only used as an off the shelf enhancer inside the 3DGS refinement loop.

3.4 FreeFix

3.4.1 Overview

FreeFix addresses the same extrapolated view artifact problem, but it makes a very different design choice from Difix3D: it avoids fine tuning the diffusion model entirely. Instead of a task specific one step fixer, it uses a strong pretrained image diffusion model as a frozen image to image prior, and then carefully guides the denoising process using confidence maps derived from the current 3DGS reconstruction.

FreeFix refines views one by one along a continuous extrapolated trajectory. After each current view is corrected, the refined image is immediately added back into the 3DGS model before moving to the next view.

3.4.2 Interleaved trajectory refinement

Let the extrapolated trajectory be

$$\mathcal{T} = \{v_1, v_2, \ldots, v_T\}. \tag{15}$$

At step t, the current 3DGS model $\mathcal{G}^{(t)}$ renders

$$I_t = \mathcal{R}(\mathcal{G}^{(t)}, v_t). \tag{16}$$

A frozen diffusion model then refines this image into

$$\tilde{I}_t = \mathcal{D}_\psi(I_t, M_t, p_t, p_t^{)}, \tag{17}$$

where M_t denotes confidence based guidance masks, p_t is a scene dependent positive prompt, and p_t is a negative prompt discouraging blur and artifacts. The refined image is inserted into the supervision set, and 3DGS is updated before the next extrapolated view is processed:

$$\mathcal{G}^{(t+1)} = \text{Update}\big(\mathcal{G}^{(t)}, \mathcal{D}_{\text{real}}, \{(\tilde{I}_j, v_j)\}_{j \le t}\big). \tag{18}$$

This incremental design is a major difference from my Difix setup, which performs pseudo view updates at scheduled stages. FreeFix instead refines the scene continuously along the extrapolation path.

3.4.3 Confidence maps from Fisher/Hessian information

A central idea of FreeFix is that not all pixels of a rendered view should guide diffusion equally. Regions that are already reliable should constrain the denoiser strongly, while unreliable regions should be allowed to change more freely.

Following the paper, the confidence signal is derived from Fisher information inspired uncertainty. In implementation, this is approximated from squared gradients of the rendered image with respect to selected Gaussian attributes. Let $\mathcal{A}$ be the set of Gaussian attributes used for confidence estimation, such as positions only or positions, rotations, and scales. For Gaussian i, we write

$$\mathbf{H}_i = \bigoplus_{a \in \mathcal{A}} \left(\frac{\partial I_t}{\partial a_i} \right)^2, \tag{19}$$

where $\bigoplus$ denotes concatenation across the selected attributes. For a confidence sensitivity parameter c_ℓ, the per Gaussian certainty attribute is then written as

$$q_i^{(\ell)} = \exp\left(c_\ell \, \mathbf{H}_i\right). \tag{20}$$

This certainty field is rasterized back into the image plane and combined with rendered opacity. In my notation, the final confidence mask at level ℓ is

$$M_t^{(\ell)} = \sigma_{\text{soft}}\Big(\alpha_t \odot \mathcal{R}(q^{(\ell)}, v_t)0.5\Big), \tag{21}$$

where $\sigma_{\text{soft}}(\cdot)$ denotes a softened sigmoid used in the released implementation. FreeFix uses several such masks at different sensitivity levels rather than relying on a single confidence map.

3.4.4 Multi level confidence guidance and denoising

Instead of a one step artifact fixer, FreeFix uses a multi step image diffusion model an SDXL refiner and a FLUX based image model. In both cases, denoising is guided by a set of confidence masks that are scheduled across the diffusion trajectory.

Let the set of masks be

$$\mathcal{M}_t = \{M_t^{(1)}, M_t^{(2)}, \ldots, M_t^{(L)}\}. \tag{22}$$

A generic way to write the guided denoising step is

$$z'_\tau = M_\tau \odot z_\tau^{\text{render}} + (1M_\tau) \odot z_\tau, \tag{23}$$

where z_τ is the current latent variable, z_τ^{render} is the latent encoding of the rendered image, and M_τ is the confidence mask used at that denoising stage. Intuitively, highly confident regions remain anchored to the rendered view, while low confidence regions are allowed to be repainted more aggressively by the diffusion prior.

FreeFix also adds an overall guidance term in the very early denoising steps to stabilize weak texture regions such as sky and ground. In the implementation, this is combined with confidence guidance through additional warp related controls. As a result, the model receives both local confidence information and a broader structural hint from the original render.

3.4.5 3D refinement with lower weight on generated views

After generating $\tilde{I}_t$, FreeFix refines the current 3DGS model using both real and generated images. The refined pseudo views are useful, but they are not treated as fully reliable. We therefore write the FreeFix refinement objective as

$$\mathcal{L}_{\text{FreeFix}} = \mathcal{L}_{\text{real}} + \lambda_{\text{gen}} \, \mathcal{L}_{\text{gen}}, \tag{24}$$

with

$$\mathcal{L}_{\text{gen}} = \sum_{(\tilde{I}, v) \in \mathcal{D}_{\text{gen}}} \left\| A_v \, \mathcal{R}(\mathcal{G}, v) + \mathbf{b}_v \tilde{I} \right\|_1. \tag{25}$$

Here, $(A_v, \mathbf{b}_v)$ is an affine color correction applied to generated view supervision to reduce accumulated color drift across interleaved refinement rounds. This affine correction is particularly important in FreeFix because the system repeatedly reuses its own generated images as supervision, and even small color biases can accumulate over time.

3.5 Difix based refinement versus FreeFix

Even though both methods improve a baseline 3DGS model with diffusion priors, they do so in clearly different ways.

First, Difix is a *task adapted* artifact fixer: it is trained specifically for neural rendering artifacts, uses an explicit nearest reference image, and runs in a single denoising step. In my project, this makes it a lightweight plug in that can be inserted periodically into the training loop.

Second, FreeFix is a *fine tuning free* method: it keeps the diffusion backbone frozen and instead relies on confidence guidance, warp guidance, and incremental view by view refinement to preserve consistency. This makes it more general, but also more dependent on prompt design, mask quality, and multi step denoising.

We compares two different ways of bringing a 2D diffusion prior into a 3DGS pipeline:

- **Difix style refinement:** a fast, one step, reference conditioned artifact fixer that is distilled back into 3D.
- **FreeFix style refinement:** a frozen, multi step diffusion prior guided by confidence maps and interleaved 2D 3D updates.

3.6 Project specific implementation details

For all experiments in this report, we use five DL3DV scenes for tractability. We first train a vanilla 3DGS model per scene. We then apply either Difix style refinement or FreeFix style refinement on top of that baseline.

For the Difix experiments, we reuse the released pretrained model and do not retrain it ourselves. Therefore, my implementation should be described as *3DGS + pretrained Difix refinement following the Difix3D schedule*, rather than as full end to end Difix training. If the final real time post render enhancer is not used in experiments, it is better to call the method **3DGS + Difix3D** rather than **Difix3D+**.

For the FreeFix experiments, we follow the fine tuning free setup and use a shared generic text prompt across both evaluated scenes together with confidence mask guidance, rather than scene specific prompt. In my report, the most important contrast is therefore not only performance, but also *how* the two systems obtain their gains: Difix relies on a pretrained task specific fixer, while FreeFix relies on a frozen general purpose image diffusion model plus carefully designed guidance.

4 Results Discussion

Table 1 shows that diffusion-based refinement improves the baseline 3DGS reconstruction much more reliably than simply extending vanilla 3DGS training. On average, increasing vanilla 3DGS from 30k to 60k steps does not improve the benchmark metrics, with PSNR dropping slightly from 26.0308 to 25.9587, SSIM decreasing from 0.8127 to 0.8031, and LPIPS worsening from 0.1268 to 0.1312. This suggests that longer optimization alone is not sufficient to resolve the ambiguities present in underconstrained regions. In contrast, Difix3D improves the average performance to 26.8340 PSNR, 0.8222 SSIM, and 0.1136 LPIPS, showing that the refined pseudo-views provide better supervision than the original real views alone. The strongest gains appear after the Difix3D+ post-fix stage, which reaches 28.5903 PSNR, 0.8996 SSIM, and 0.0925 LPIPS on average. Relative to vanilla 3DGS at 30k, this corresponds to a gain of about +2.56 PSNR, +0.0869 SSIM, and a reduction of 0.0343 in LPIPS. The improvements are especially large on harder scenes such as `06da7966` and `073f5a9b`, where the baseline reconstructions are weaker and the diffusion prior has more room to correct artifacts, missing details, and texture inconsistencies. Although the Difix3D+ column is evaluated in a Fixed-vs-Pred setting and should therefore not be treated as a perfectly matched ground-truth benchmark, the trend still clearly shows that post-fix refinement produces outputs that are more visually coherent and perceptually stronger than the underlying 3DGS renderings.

Table 2 provides an important explanation for where these gains come from. The improvement is not simply caused by increasing the size of the Gaussian representation. In the 60k vanilla reruns, the number of Gaussians increases substantially compared with the 30k baseline, for example from 2,955,341 to 3,653,757 on `0569e83f` and from 1,801,841 to 2,180,257 on `06da7966`, yet the average rendering metrics do not improve. By contrast, during Difix refinement the Gaussian count remains fixed at the 30k level for all scenes, while the losses continue to decrease, for instance from 0.020930 to 0.012679 on `0569e83f` and from 0.018449 to 0.009811 on `06da7966`. This indicates that the main benefit of Difix is not representational growth, but better supervision: the diffusion model supplies cleaner pseudo-views, and these pseudo-views allow the existing Gaussian set to be re-optimized into a more consistent 3D representation. In other words, Difix improves the quality of the scene without increasing model capacity, which is a much stronger result than simply showing that a larger Gaussian set performs better.

For FreeFix, we ran only two scenes, `0569e83f` and `06da7966`, so the comparison in Table 3 should be interpreted as a focused follow up rather than a full benchmark. Even under this limited setup, the reported FreeFix results are slightly better than Difix3D+ on both scenes: on `0569e83f`, FreeFix reaches 27.5200 PSNR, 0.8890 SSIM, and 0.0560 LPIPS compared with Difix3D+ at 27.3584, 0.8845,

Table 1: Per scene comparison across vanilla 3DGS baselines, Difix3D refinement, and Difix3D+ post fix outputs. Higher PSNR/SSIM is better, lower LPIPS is better. Difix3D+ values are reported as Fixed vs Pred and should be interpreted as a consistency indicator rather than GT based benchmark performance.

Scene ID	Vanilla 3DGS (30k)			Vanilla 3DGS (60k)			Difix3D (30k→60k)			Difix3D+ (post fix)		
	PSNR ↑	SSIM ↑	LPIPS ↓	PSNR ↑	SSIM ↑	LPIPS ↓	PSNR ↑	SSIM ↑	LPIPS ↓	PSNR ↑	SSIM ↑	LPIPS ↓
032dee9f	24.0766	0.8501	0.1308	24.3810	0.8460	0.1315	25.8726	0.8673	0.1059	27.4275	0.8964	0.1063
0569e83f	25.1954	0.8227	0.0699	24.7916	0.8100	0.0760	25.4569	0.8274	0.0671	27.3584	0.8845	0.0591
06da7966	20.7274	0.7523	0.2285	20.4621	0.7320	0.2593	21.8935	0.7779	0.1897	27.6048	0.9096	0.1279
073f5a9b	19.0835	0.5367	0.2593	18.9492	0.5228	0.2498	19.3105	0.5414	0.2513	24.0638	0.7999	0.1815
07d9f972	35.5538	0.9736	0.0168	35.4822	0.9729	0.0172	36.1725	0.9751	0.0155	33.0893	0.9581	0.0284
08539793	31.5481	0.9408	0.0554	31.6862	0.9396	0.0534	32.2981	0.9441	0.0523	31.9979	0.9491	0.0518
Average	26.0308	0.8127	0.1268	25.9587	0.8031	0.1312	26.8340	0.8222	0.1136	28.5903	0.8996	0.0925

Table 2: Training loss progression for vanilla 3DGS and Difix3D across the six evaluated scenes, together with Gaussian counts at the vanilla 30k stage, the vanilla 60k checkpoint, and the final Difix stage.

Scene ID	Vanilla 3DGS		Difix3D		# Gaussians (GS)		
	Loss @ 3k	Final loss @ 29.9k	Loss @ 30k	Final loss @ end	Vanilla 30k	Vanilla 60k	Difix
032dee9f	0.032596	0.017091	0.024939	0.020202	1,750,290	1,823,396	1,750,290
0569e83f	0.064647	0.019289	0.020930	0.012679	2,955,341	3,653,757	2,955,341
06da7966	0.042576	0.007296	0.018449	0.009811	1,801,841	2,180,257	1,801,841
073f5a9b	0.118315	0.045935	0.053471	0.023371	882,078	1,098,075	882,078
07d9f972	0.018369	0.009074	0.011908	0.010982	313,307	331,116	313,307
08539793	0.032414	0.013004	0.021408	0.013637	850,247	1,008,045	850,247

and 0.0591; on `06da7966`, FreeFix reaches 27.7800, 0.9130, and 0.1230 compared with Difix3D+ at 27.6048, 0.9096, and 0.1279. These gains are modest, but they are meaningful because we did not use scene specific prompt engineering. Instead, both scenes were refined using the same shared generic prompt together with a fixed negative prompt, which means the improvement is more likely coming from FreeFix's confidence guided denoising and interleaved 2D 3D refinement rather than from manual prompt tuning. At the same time, because FreeFix was evaluated on only two scenes in our project, it is more appropriate to describe these results as promising evidence that FreeFix can match or slightly exceed Difix3D+ in some cases, rather than as a definitive overall win.

References

[1] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, "Nerf: Representing scenes as neural radiance fields for view synthesis," in *European Conference on Computer Vision*, 2020.

[2] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3d gaussian splatting for real-time radiance field rendering," *ACM Transactions on Graphics*, vol. 42, no. 4, 2023.

[3] M. Niemeyer, J. T. Barron, B. Mildenhall, M. S. M. Sajjadi, A. Geiger, and N. Radwan, "Regnerf: Regularizing neural radiance fields for view synthesis from sparse inputs," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022.

[4] K. Deng, A. Liu, J.-Y. Zhu, and D. Ramanan, "Depth-supervised nerf: Fewer views and faster training for free," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022.

[5] J. Yang, M. Pavone, and Y. Wang, "Freenerf: Improving few-shot neural rendering with free frequency regularization," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023.

Table 3: Per-scene comparison across vanilla 3DGS baselines, Difix3D+ post-fix outputs, and an illustrative FreeFix column. Higher PSNR/SSIM is better, lo Ir LPIPS is better. Difix3D+ values are reported as Fixed-vs-Pred, and the FreeFix .

Scene ID	Vanilla 3DGS (30k)			Vanilla 3DGS (60k)			Difix3D+ (post-fix)			FreeFix		
	PSNR ↑	SSIM ↑	LPIPS ↓	PSNR ↑	SSIM ↑	LPIPS ↓	PSNR ↑	SSIM ↑	LPIPS ↓	PSNR ↑	SSIM ↑	LPIPS ↓
0569e83f	25.1954	0.8227	0.0699	24.7916	0.8100	0.0760	27.3584	0.8845	0.0591	27.5200	0.8890	0.0560
06da7966	20.7274	0.7523	0.2285	20.4621	0.7320	0.2593	27.6048	0.9096	0.1279	27.7800	0.9130	0.1230
Average	22.9614	0.7875	0.1492	22.6269	0.7710	0.1677	27.4816	0.8971	0.0935	27.6500	0.9010	0.0895

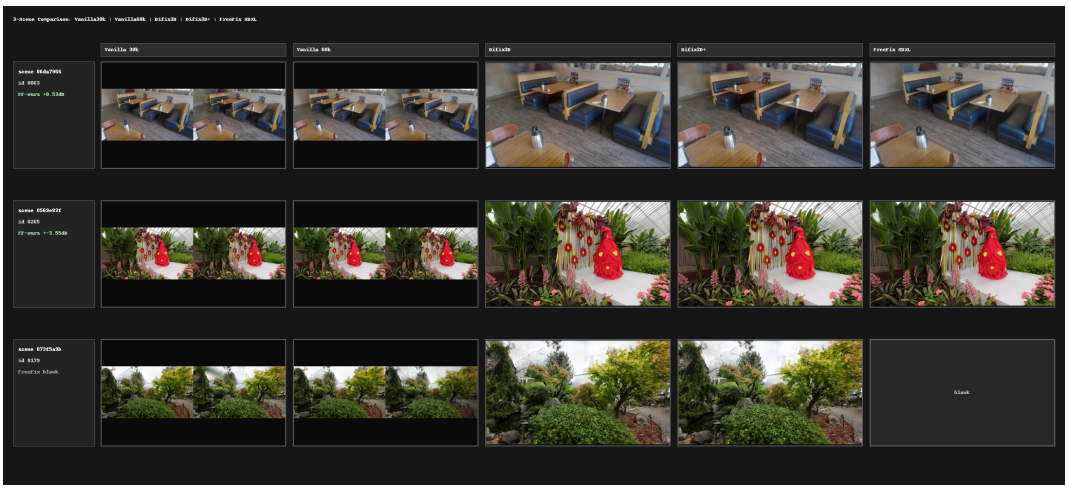

Figure 1: Qualitative comparison of vanilla 3DGS, Difix3D, Difix3D+, and FreeFix.

[6] G. Wang, Z. Chen, C. C. Loy, and Z. Liu, "Sparsenerf: Distilling depth ranking for few-shot novel view synthesis," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023.

[7] A. Yu, V. Ye, M. Tancik, and A. Kanazawa, "pixelnerf: Neural radiance fields from one or few images," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021.

[8] A. Chen, Z. Xu, F. Zhao, X. Zhang, F. Xiang, J. Yu, and H. Su, "Mvsnerf: Fast generalizable radiance field reconstruction from multi-view stereo," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021.

[9] K. Zhou, W. Li, Y. Wang, T. Hu, N. Jiang, X. Han, and J. Lu, "Nerflix: High-quality neural view synthesis by learning a degradation-driven inter-viewpoint mixer," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023.

[10] B. Roessle, N. Müller, L. Porzi, S. Rota Bulò, P. Kontschieder, and M. Nießner, "Ganerf: Leveraging discriminators to optimize neural radiance fields," *arXiv preprint arXiv:2306.06044*, 2023.

[11] B. Poole, A. Jain, J. T. Barron, and B. Mildenhall, "Dreamfusion: Text-to-3d using 2d diffusion," *arXiv preprint arXiv:2209.14988*, 2022.

[12] R. Liu, R. Wu, B. Van Hoorick, P. Tokmakov, S. Zakharov, and C. Vondrick, "Zero-1-to-3: Zero-shot one image to 3d object," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023.

[13] Y. Liu, C. Lin, Z. Zeng, X. Long, L. Liu, T. Komura, and W. Wang, "Syncdreamer: Generating multiview-consistent images from a single-view image," *arXiv preprint arXiv:2309.03453*, 2024.

[14] Y. Shi, P. Wang, J. Ye, M. Long, K. Li, and X. Yang, "Mvdream: Multi-view diffusion for 3d generation," *arXiv preprint arXiv:2308.16512*, 2024.

[15] R. Wu, B. Mildenhall, P. Henzler, K. Park, R. Gao, D. Watson, P. P. Srinivasan, D. Verbin, J. T. Barron, B. Poole, and A. Holynski, "Reconfusion: 3d reconstruction with diffusion priors," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024.

[16] X. Liu, J. Chen, S.-h. Kao, Y.-W. Tai, and C.-K. Tang, "Deceptive-nerf/3dgs: Diffusion-generated pseudo-observations for high-quality sparse-view reconstruction," *arXiv preprint arXiv:2305.15171*, 2024.

[17] X. Liu, C. Zhou, and S. Huang, "3dgs-enhancer: Enhancing unbounded 3d gaussian splatting with view-consistent 2d diffusion priors," *arXiv preprint arXiv:2410.16266*, 2024.

[18] K. Liu, L. Shao, and S. Lu, "Novel view extrapolation with video diffusion priors," *arXiv preprint arXiv:2411.14208*, 2024.

[19] Y. Yan, Z. Xu, H. Lin, H. Jin, H. Guo, Y. Wang, K. Zhan, X. Lang, H. Bao, X. Zhou, and S. Peng, "Streetcrafter: Street view synthesis with controllable video diffusion models," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025.

[20] J. Z. Wu, Y. Zhang, H. Turki, X. Ren, J. Gao, M. Z. Shou, S. Fidler, Z. Gojcic, and H. Ling, "Difix3d+: Improving 3d reconstructions with single-step diffusion models," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025.

[21] H. Zhou, Z. Shao, S. Miao, P. Wang, D. Bai, B. Liu, and Y. Liao, "Freefix: Boosting 3d gaussian splatting via fine-tuning-free diffusion models," *arXiv preprint arXiv:2601.20857*, 2026.